

Section A - Conversions

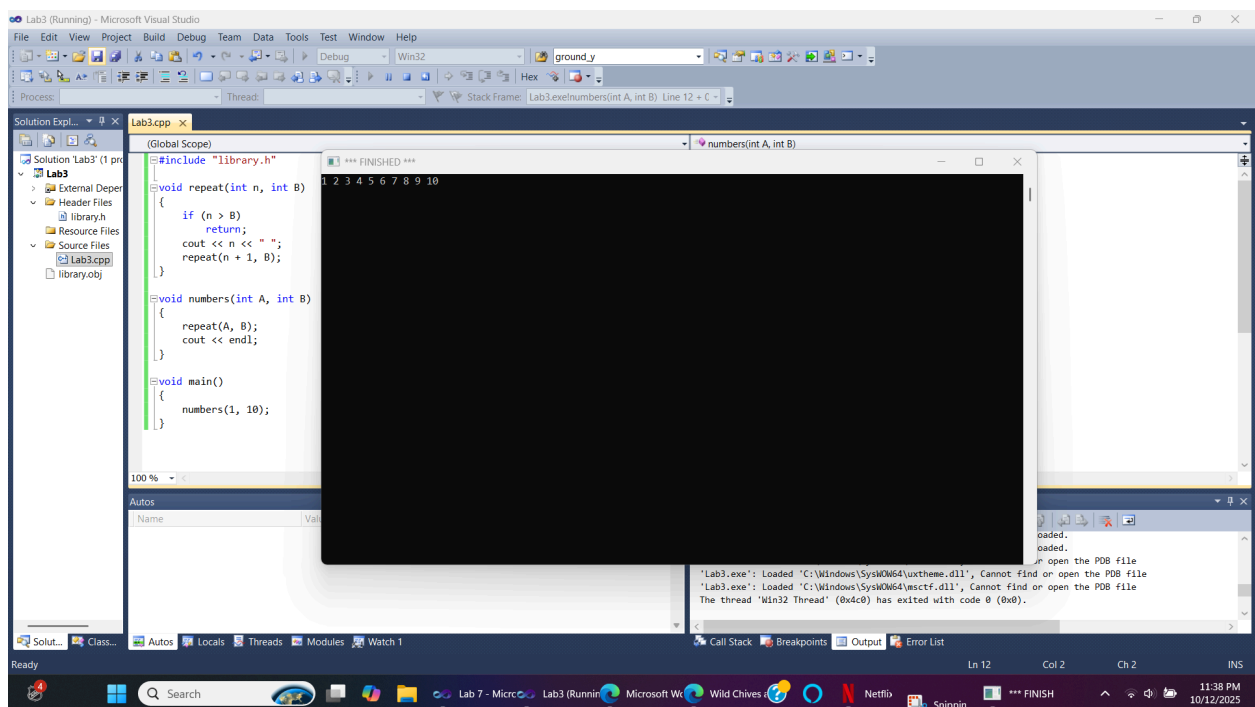
A1. Remembering how to start

```
#include "library.h"
```

```
void repeat(int n, int B)
{
    if (n > B)
        return;
    cout << n << " ";
    repeat(n + 1, B);
}
```

```
void numbers(int A, int B)
{
    repeat(A, B);
    cout << endl;
}
```

```
void main()
{
    numbers(1, 10);
}
```



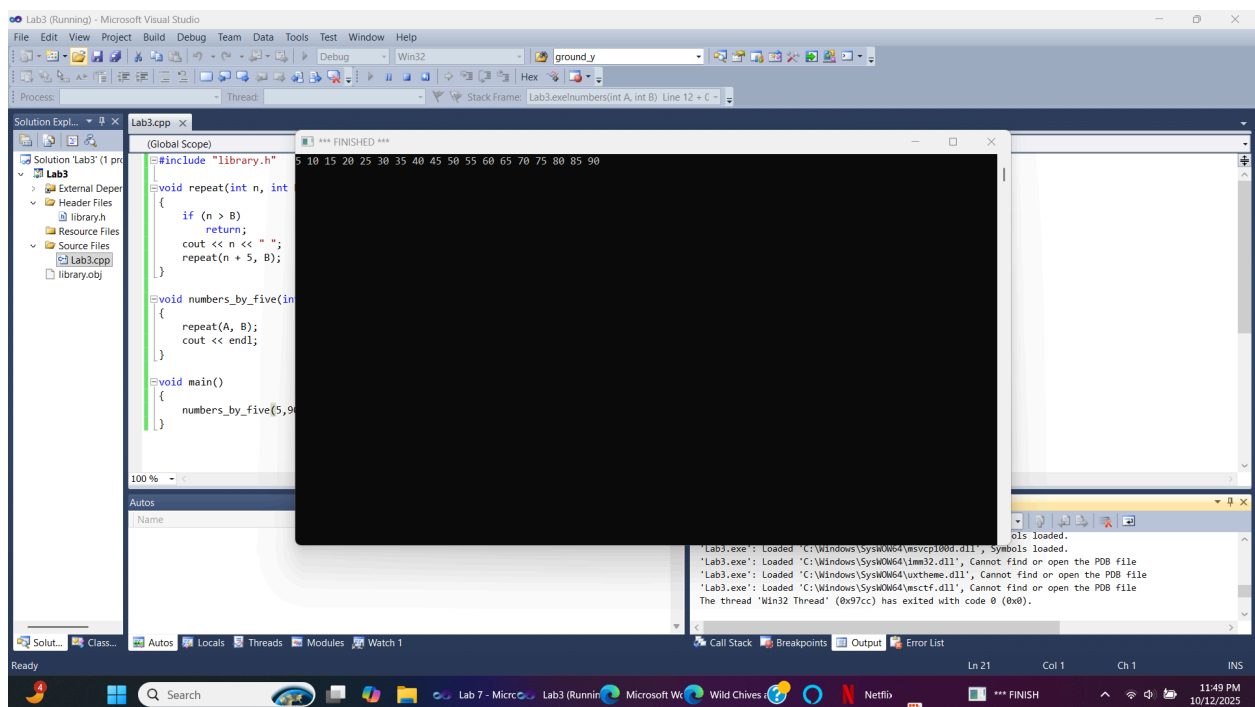
A2. Angles for Computers

```
#include "library.h"
```

```
void repeat(int n, int B)
{
    if (n > B)
        return;
    cout << n << " ";
    repeat(n + 5, B);
}
```

```
void numbers_by_five(int A, int B)
{
    repeat(A, B);
    cout << endl;
}
```

```
void main()
{
    numbers_by_five(5,90);
}
```



```

#include "library.h"

double pi = 3.14159265359;

void deg_to_rad(int start, int end)
{
    if (start > end)
        return;

    double rad = start * pi / 180;

    double round_by_2 = rad * 100;
    int round = (int)(round_by_2 + 0.5);
    rad = round / 100.0;

    cout << start << " degrees is " << rad << " radians" << endl;

    deg_to_rad(start + 5, end);
}

void rad_to_deg(double rad)
{
    if (rad > 1)
        return;

    double deg = rad * 180 / pi;
    int deg_round = (int)(deg + 0.5);

    double round_by_2 = rad * 10;
    int round = (int)(round_by_2 + 0.5);
    double rad_rounded = round / 10.0;

    cout << rad_rounded << " radians is " << deg_round << " degrees" << endl;

    rad_to_deg(rad + 0.1);
}

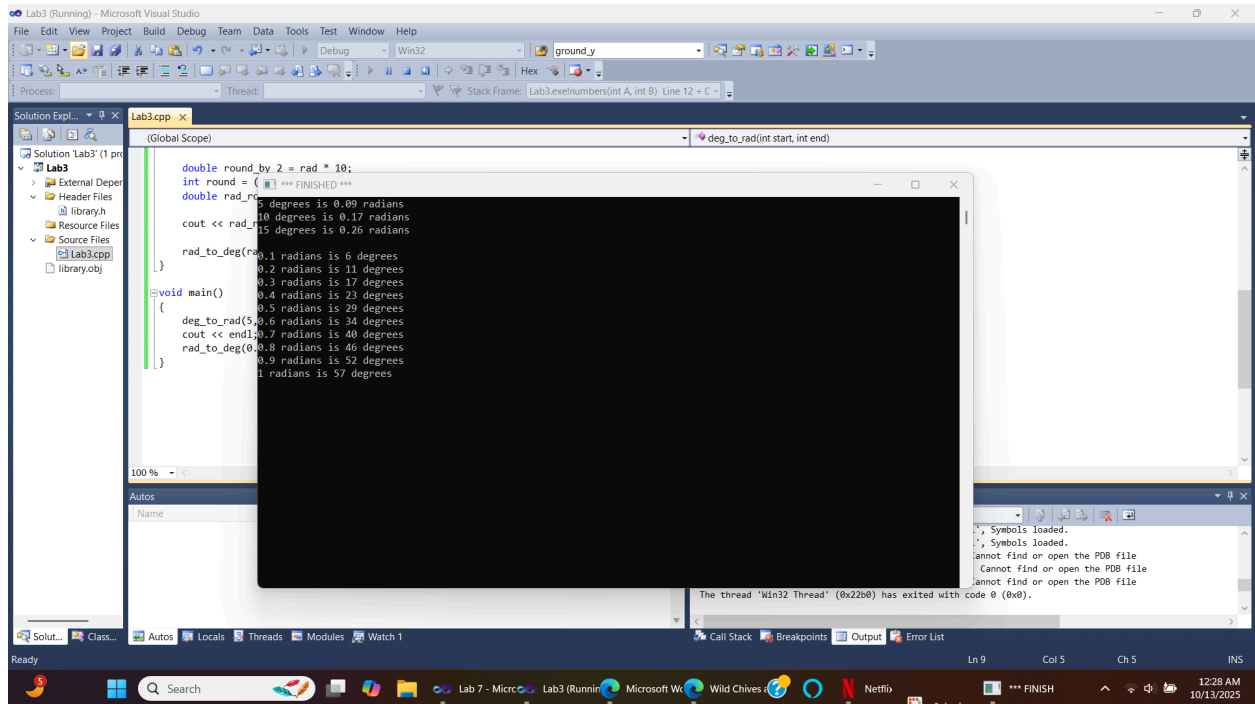
void main()

```

```

{
    deg_to_rad(5, 15);
    cout << endl;
    rad_to_deg(0.1);
}

```



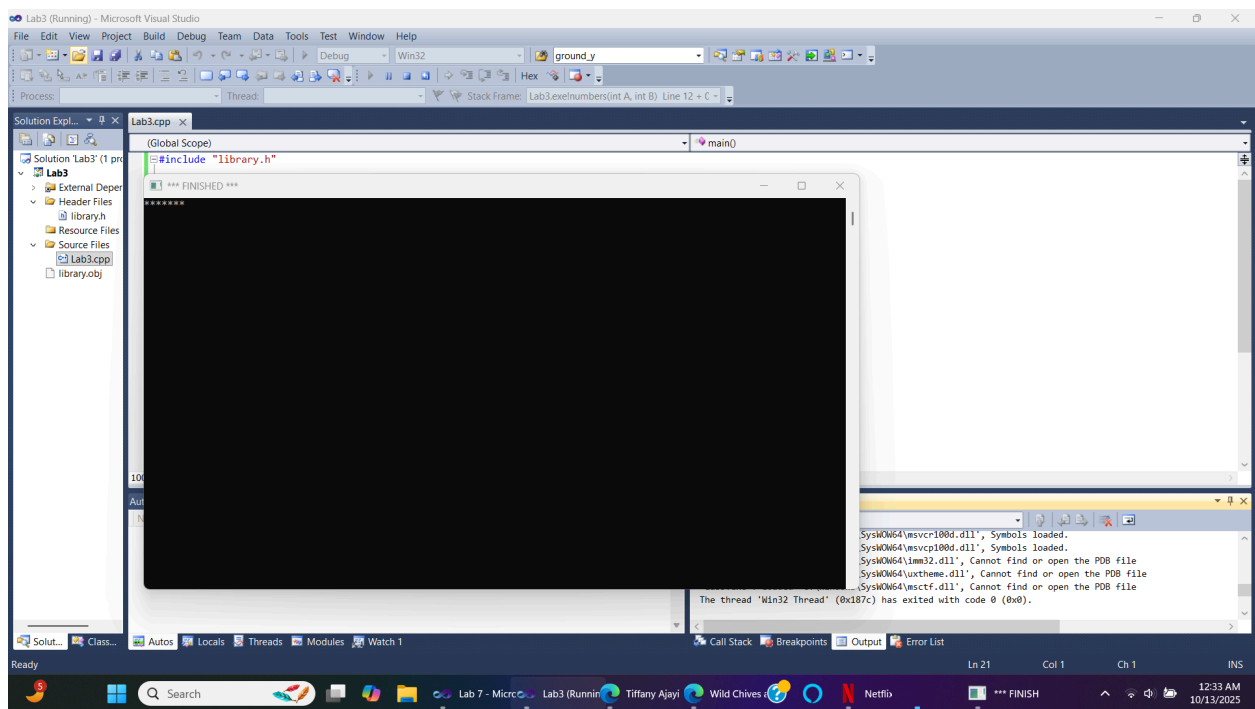
Section B - ASCII Art

B1. Stars

```
#include "library.h"
```

```
void stars(int N)
{
    if (N <= 0) return;
    cout << "**";
    stars(N - 1);
}
```

```
void main()
{
    stars(7);
}
```

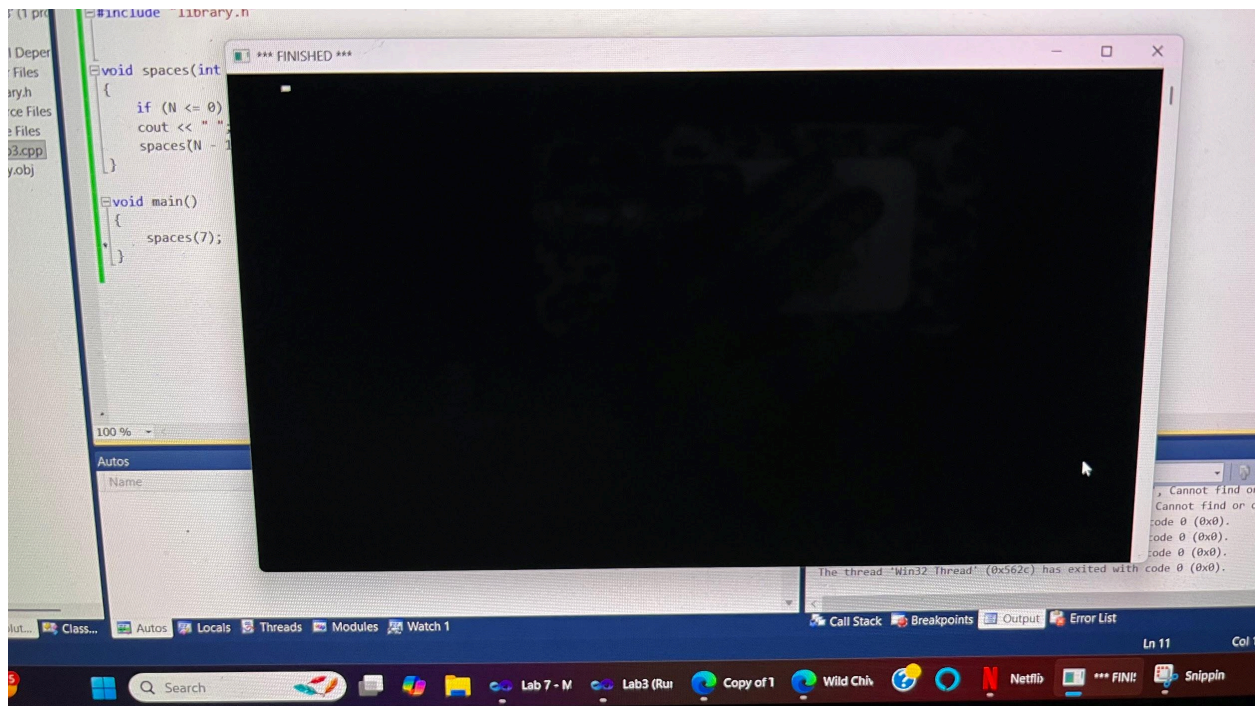


B2. Spaces

```
#include "library.h"
```

```
void spaces(int N)
{
    if (N <= 0) return;
    cout << " ";
    spaces(N - 1);
}
```

```
void main()
{
    spaces(7);
}
```



B3. Stars and Dots

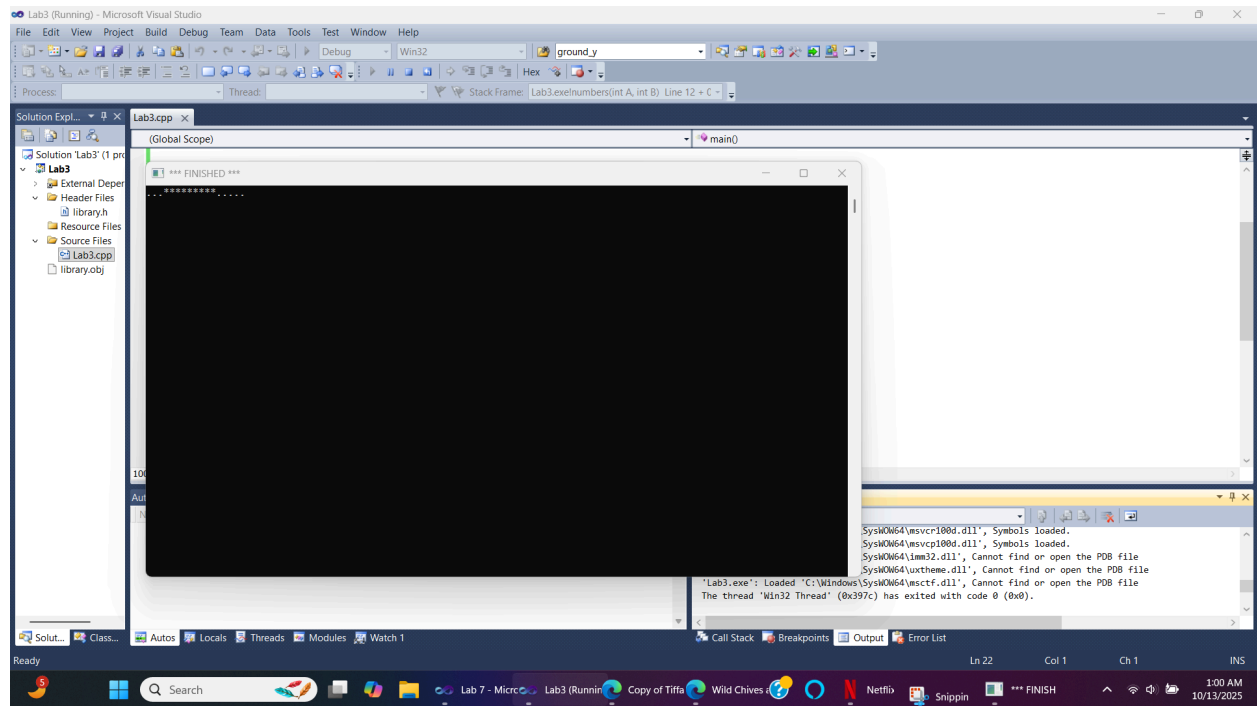
```
#include "library.h"
```

```
void dots(int N)
{
    if (N <= 0) return;
    cout << ".";
    dots(N - 1);
}
```

```
void stars(int N)
{
    if (N <= 0) return;
    cout << "***";
    stars(N - 1);
}
```

```
void dots_stars_dots(int A, int B, int C)
{
    dots(A);
    stars(B);
    dots(C);
    cout << endl;
}
```

```
void main()
{
    dots_stars_dots(3, 9, 5);
}
```

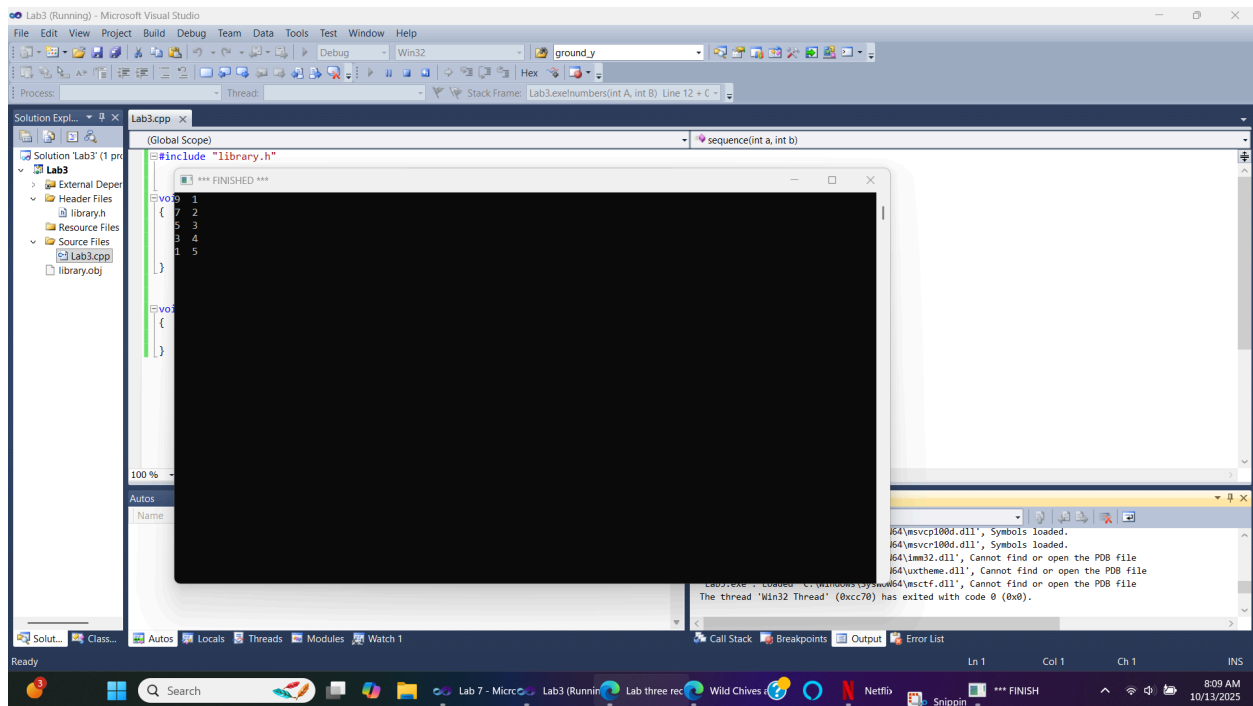


B4. Another Adaption

```
#include "library.h"
```

```
void sequence(int a, int b)
{
    if (a <= 0) return;
    cout << a << " " << b << endl;
    sequence(a - 2, b + 1);
}
```

```
void main()
{
    sequence(9, 1);
}
```



B5. Combining

```
#include "library.h"
```

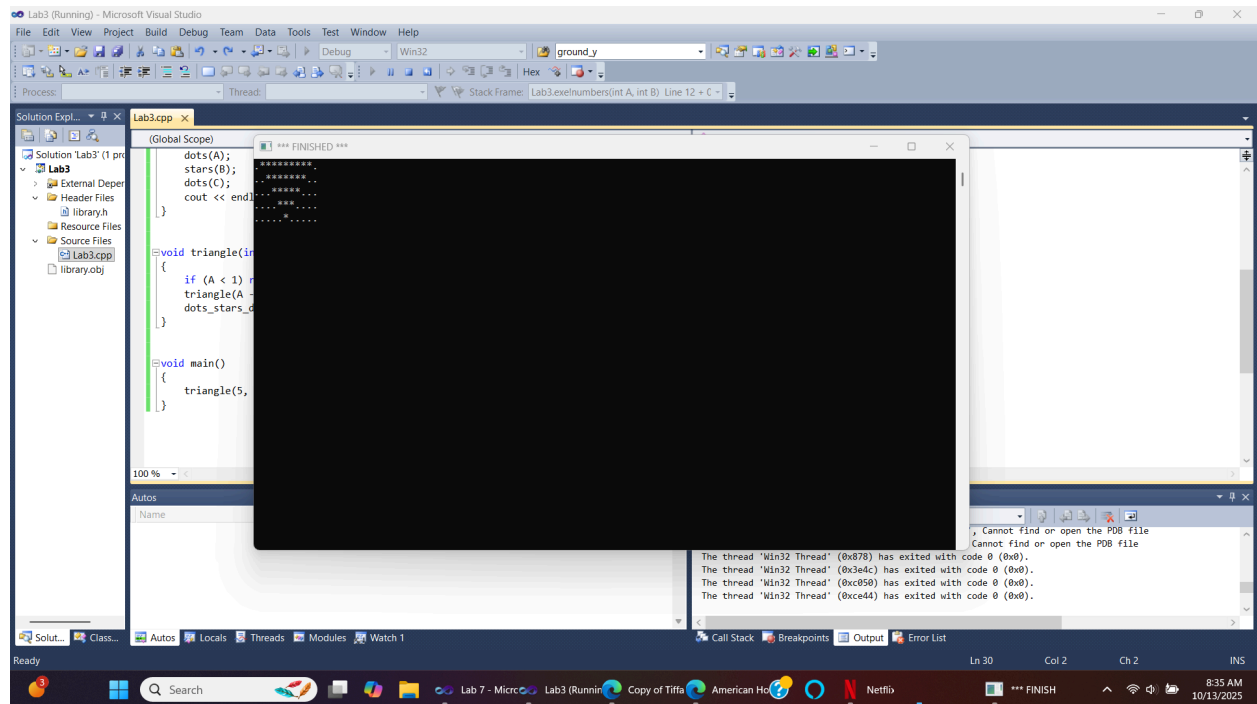
```
void dots(int N)
{
    if (N <= 0) return;
    cout << ".";
    dots(N - 1);
}
```

```
void stars(int N)
{
    if (N <= 0) return;
    cout << "**";
    stars(N - 1);
}
```

```
void dots_stars_dots(int A, int B, int C)
{
    dots(A);
    stars(B);
    dots(C);
    cout << endl;
}
```

```
void triangle(int A, int B)
{
    if (A < 1) return;
    triangle(A - 1, B + 2);
    dots_stars_dots(A, B, A);
}
```

```
void main()
{
    triangle(5, 1);
}
```



B6. Up-side Down

```
#include "library.h"
```

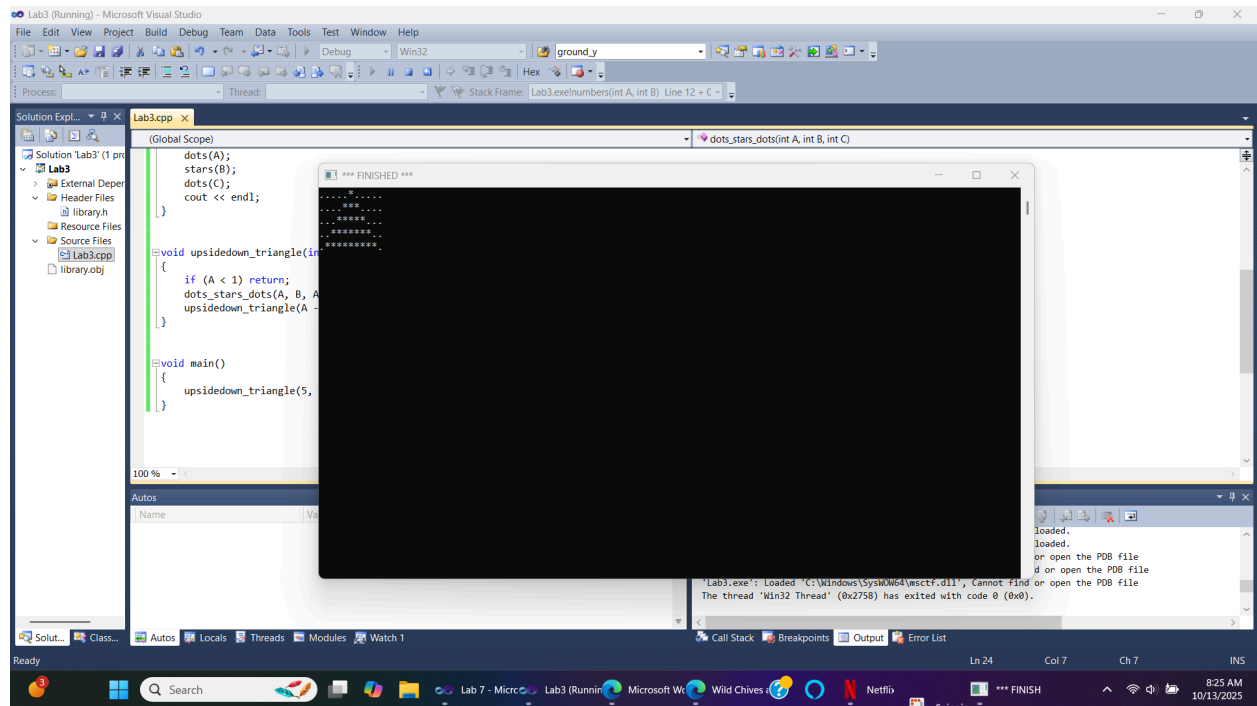
```
void dots(int N)
{
    if (N <= 0) return;
    cout << ".";
    dots(N - 1);
}
```

```
void stars(int N)
{
    if (N <= 0) return;
    cout << "**";
    stars(N - 1);
}
```

```
void dots_stars_dots(int A, int B, int C)
{
    dots(A);
    stars(B);
    dots(C);
    cout << endl;
}
```

```
void upsidedown_triangle(int A, int B)
{
    if (A < 1) return;
    dots_stars_dots(A, B, A);
    upsidedown_triangle(A - 1, B + 2);
}
```

```
void main()
{
    upsidedown_triangle(5, 1);
}
```



B7. Diamond

```
#include "library.h"
```

```
void dots(int N)
{
    if (N <= 0) return;
    cout << ".";
    dots(N - 1);
}
```

```
void stars(int N)
{
    if (N <= 0) return;
    cout << "**";
    stars(N - 1);
}
```

```
void dots_stars_dots(int A, int B, int C)
{
    dots(A);
    stars(B);
    dots(C);
    cout << endl;
}
```

```
void triangle(int A, int B)
{
    if (A < 1) return;
    triangle(A - 1, B + 2);
    dots_stars_dots(A, B, A);
}
```

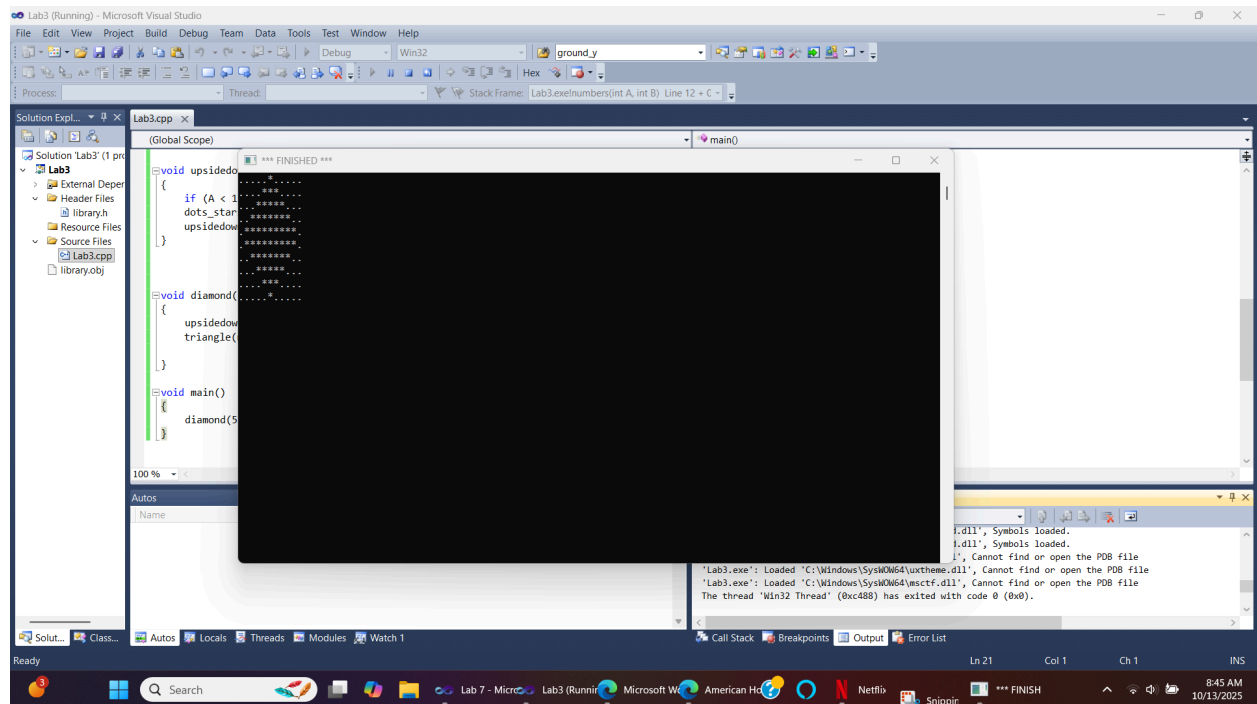
```
void upsidedown_triangle(int A, int B)
{
    if (A < 1) return;
    dots_stars_dots(A, B, A);
    upsidedown_triangle(A - 1, B + 2);
}
```

```

void diamond(int n)
{
    upsidedown_triangle(n,1);
    triangle(n, 1);
}

void main()
{
    diamond(5);
}

```



Section C - Circles

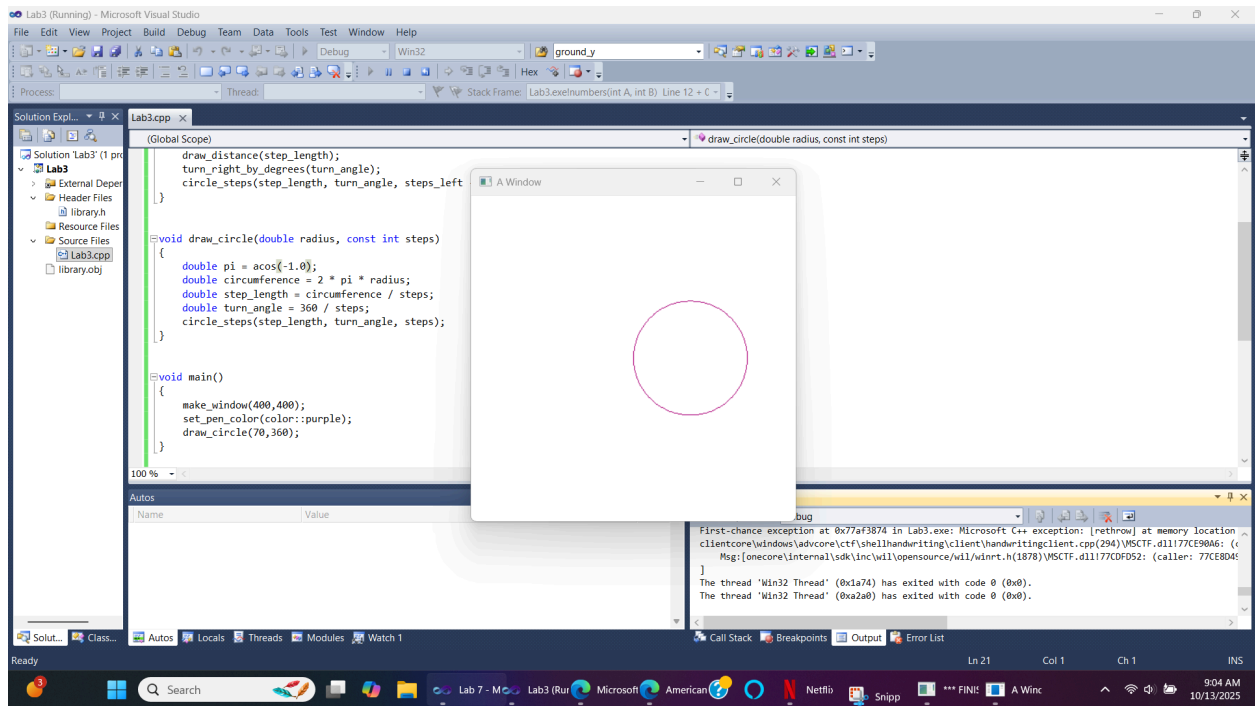
C1. A circle

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    if (steps_left == 0)
    {
        return;
    }
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(double radius, const int steps)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference / steps;
    double turn_angle = 360 / steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void main()
{
    make_window(400,400);
    set_pen_color(color::purple);
    draw_circle(70,360);
}
```



C2. Weaponizing

```
#include "library.h"
```

```
void circle_steps(double step_length, double turn_angle, int steps_left)
{
    if (steps_left == 0)
    {
        return;
    }
    draw_distance(step_length);
    turn_right_by_degrees(turn_angle);
    circle_steps(step_length, turn_angle, steps_left - 1);
}
```

```
void draw_circle(double radius, const int steps)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference / steps;
    double turn_angle = 360 / steps;
    circle_steps(step_length, turn_angle, steps);
}
```

```
void draw_cannon(double xg, double yg, double a)
{
    double r = 30;
    double L1 = 40;
    double L2 = 80;
    double w1 = 40;
    double w2 = 20;

    double xc = xg;
    double yc = yg - r;

    move_to(xc, yc);
    draw_circle(r, 60);

    double b = asin((w1 - w2)/2/(L1 + L2));
```

```

double xp = xc - L1 * sin(a-b);
double yp = yc + L1 * cos(a-b);

double len = (L1 + L2) * cos(b);

double d = sqrt(len*len + (w1*w1)/4);
double g = asin((w1/2)/d);
double xe = xp + d * sin(a-g);
double ye = yp - d * cos(a-g);

double phi = a - g;

double px = cos(phi);
double py = sin(phi);

double back_top_x = xp + (w1/2) * px;
double back_top_y = yp + (w1/2) * py;
double back_bot_x = xp - (w1/2) * px;
double back_bot_y = yp - (w1/2) * py;

double front_top_x = xe + (w2/2) * px;
double front_top_y = ye + (w2/2) * py;
double front_bot_x = xe - (w2/2) * px;
double front_bot_y = ye - (w2/2) * py;

move_to(back_bot_x, back_bot_y);
draw_to(front_bot_x, front_bot_y);
draw_to(front_top_x, front_top_y);
draw_to(back_top_x, back_top_y);
draw_to(back_bot_x, back_bot_y);
}

void main()
{
    make_window(700,500);
    set_pen_color(color::purple);
    draw_cannon(200,380,45);
    draw_cannon(400,380,90);
}

```

